Data 100, Summer 2026

Discussion #2 Solutions

Note: Your TA will probably not cover all the problems. This is totally fine; the discussion worksheets are not designed to be finished within an hour. They are deliberately made slightly longer so they can serve as resources you can use to practice, reinforce, and build upon concepts discussed in lectures, labs, and homework.

EDA Practice

The following questions are based on the `elections.csv` data referenced in the lecture. Assume that you have run the loaded data into the `elections` variable as follows:

```
elections = pd.read_csv('elections.csv')
```

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073
4	1832	Andrew Jackson	Democratic	702735	win	54.574789
...	...	...	...	...	...	...
182	2024	Donald Trump	Republican	77303568	win	49.808629
183	2024	Kamala Harris	Democratic	75019230	loss	48.336772
184	2024	Jill Stein	Green	861155	loss	0.554864
185	2024	Robert Kennedy	Independent	756383	loss	0.487357
186	2024	Chase Oliver	Libertarian Party	650130	loss	0.418895

187 rows × 6 columns

For the scenarios below, write an analysis plan that starts with "For each..."

1. How many presidential election years are included in this dataset?

Solution:

"**For each** unique value of `Year`, **count** one presidential election."

"**Count** the number of unique years."

The other columns do not matter because each unique `Year` represents one presidential election.

Pandas Code

```
len(elections['Year'].unique())
```

SQL Query

```
SELECT COUNT(DISTINCT Year) AS n_elections
FROM elections;
```

2. Across all elections, which parties have the most candidate appearances?

Solution:

"**For each** unique value of `Party`, **count** the number of rows."

"**Sort** the data in descending order of the number of rows."

The other columns do not matter because each row represents one candidate in one election.

Pandas Code

```
elections.groupby('Party').size().sort_values(ascending=False)
```

SQL Query

```
SELECT Party, COUNT(*) AS n_candidates
FROM elections
GROUP BY Party
ORDER BY n_candidates DESC;
```

3. From 1900 through 1999, how many times did a Democrat run for president?

Solution:

"**For each** row, keep it if `Year` is in the 1900s and `Party` is `Democratic`."

"**Count** the number of remaining rows."

Each remaining row represents one time a Democrat ran for president.

Pandas Code

```
elections[
    (elections['Year'] >= 1900) &
    (elections['Year'] < 2000) &
    (elections['Party'] == 'Democratic')
].shape[0]
```

SQL Query

```
SELECT COUNT(*) AS n_democrats
FROM elections
WHERE Year >= 1900
  AND Year < 2000
  AND Party = 'Democratic';
```

4. Was there ever a year when one party ran more than one candidate?

Solution:

"**For each** unique combination of `Year` and `Party`, **count** the number of rows."

"**Keep** only the groups where the number of rows is greater than 1."

"**Check** whether any groups remain."

If any groups remain, then yes, there was a year when one party ran more than one candidate.

Pandas Code

```
grouped = (
            elections.groupby(['Year', 'Party']).size()
            .reset_index(name='n_candidates')
)

grouped[grouped['n_candidates'] > 1]
```

SQL Query

```
SELECT Year, Party, COUNT(*) AS n_candidates
FROM elections
GROUP BY Year, Party
HAVING COUNT(*) > 1;
```

5. In how many elections did at least one candidate receive less than 5% of the popular vote?

Solution:

"**For each** unique value of `Year`, **check** whether at least one candidate had less than 5 %."

"**Count** the number of years where this happened."

We count unique `Year` values, not rows, because the question asks how many elections.

Pandas Code

```
len(elections[elections['%'] < 5]['Year'].unique())
```

SQL Query

```
SELECT COUNT(DISTINCT Year) AS n_elections
FROM elections
WHERE "%" < 5;
```

6. What does the `elections` data look like for parties that have ever gotten more than 50% of the vote?

Solution:

Solution:

"**For each** unique value of `Party`, check whether the largest value of `%` is greater than 50."

"**Keep** only the parties where this condition is true."

The result is a filtered dataframe containing all rows for parties that have ever gotten more than 50% of the vote.

Pandas Code

```
filtered_parties = (
                    elections.groupby("Party")
                    .filter(lambda sf: sf["%"].max() > 50)
)
```

SQL Query

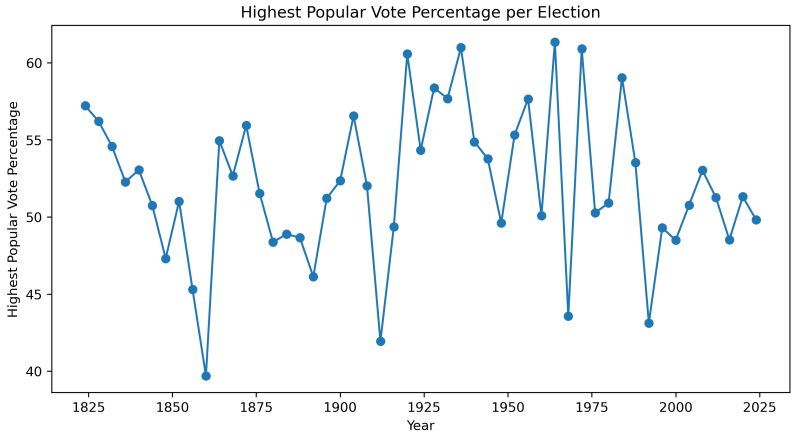
```
SELECT *
FROM elections
WHERE Party IN (
    SELECT Party
    FROM elections
    GROUP BY Party
    HAVING MAX("%") > 50
);
```

Visualizations

Using the same data set above, sketch a visualization corresponding each of the questions below. Then, write an analysis plan to create a dataframe that could be used to produce to the visualization.

1. What was the highest percentage of the popular vote received by any candidate during each election?

Solution:



"**For each** unique value of `Year`, find the largest value of `%`."

"This gives the highest popular vote percentage received by any candidate in that election."

"**Create** a new dataframe with one row per election year and one column called `max_percent`."

"**Sort** the dataframe in increasing order of `Year`."

"**Plot** `Year` against `max_percent` using a line plot."

Pandas Code

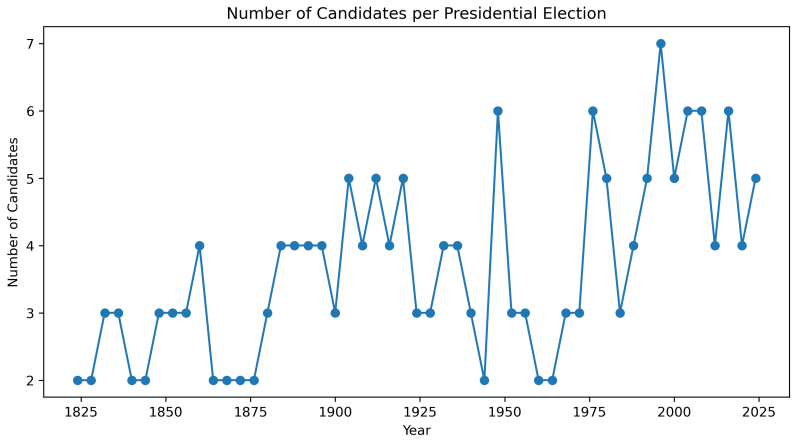
```
import matplotlib.pyplot as plt

highest_percent_per_election = (
    elections.groupby('Year', as_index=False)['%']
    .max()
    .rename(columns={'%': 'max_percent'})
    .sort_values('Year')
)
```

SQL Query

```
SELECT Year, MAX("%") AS max_percent
FROM elections
GROUP BY Year
ORDER BY Year;
```

2. What is the trend in the number of candidates per election?

Solution:



"**For each** unique value of `Year`, **count** the number of rows."

"Each row represents one candidate running in that election."

"**Create** a dataframe with one row per election year and columns Year and n_candidates."

"**Sort** the dataframe in increasing order of `Year`."

Pandas Code

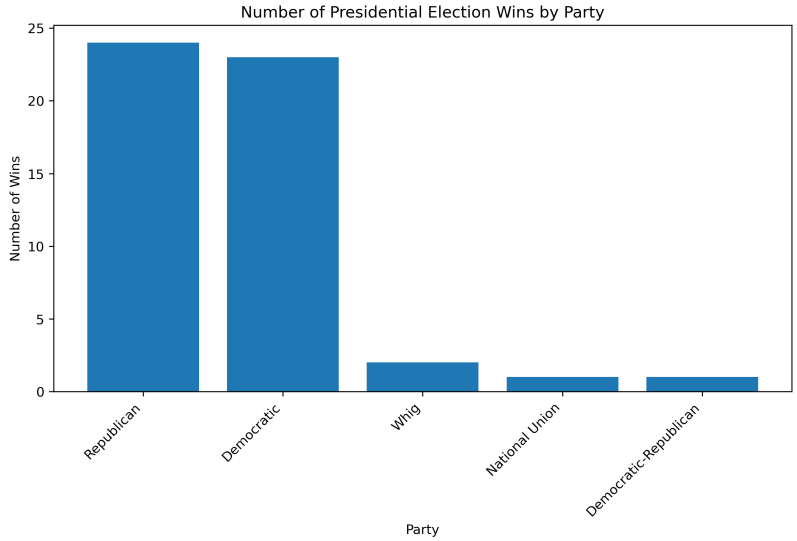
```
candidates_per_election = (
    elections.groupby('Year')
    .size()
    .reset_index(name='n_candidates')
    .sort_values('Year')
)
```

SQL Query

```
SELECT Year, COUNT(*) AS n_candidates
FROM elections
GROUP BY Year
ORDER BY Year;
```

3. Which party has won the most elections?

Solution:



"**For each** row, keep only the rows where the candidate won the election."

"**For each** unique value of `Party` in the remaining data, **count** the number of rows."

"**Sort** the dataframe in descending order of the number of wins."

The grouped count represents the total number of elections won by each party.

Pandas Code

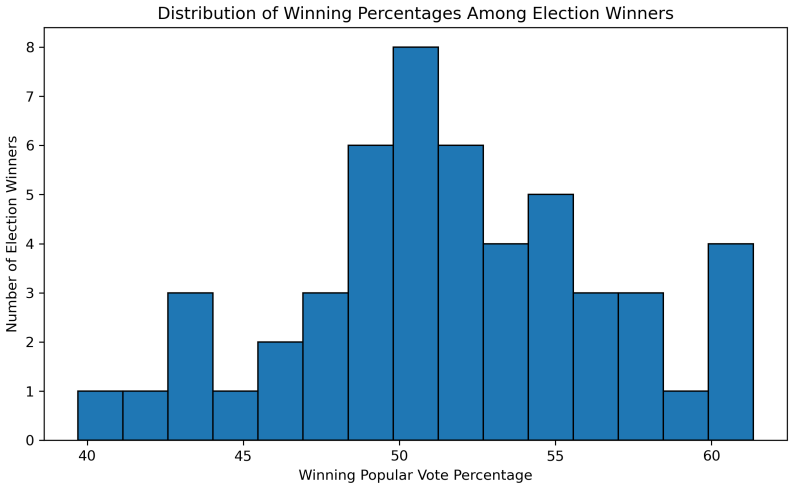
```
wins_by_party = (
    elections[elections['Result'] == 'win']
    .groupby('Party')
    .size()
    .reset_index(name='n_wins')
    .sort_values('n_wins', ascending=False)
)
```

SQL Query

```
SELECT Party, COUNT(*) AS n_wins
FROM elections
WHERE Result = 'win'
GROUP BY Party
ORDER BY n_wins DESC;
```

4. What does the distribution of winning percentages among all election winners look like?

Solution:



"**For each** row, keep only the rows where the candidate won the election."

"**For each** winning candidate, use the value in the `%` column."

"Create a dataframe with one row per election winner and columns `Year`, `Candidate`, and `%`."

The histogram shows how common different winning popular vote percentages are among election winners.

Pandas Code

```
winning_percentages = (
    elections[elections['Result'] == 'win']
    [['Year', 'Candidate', '%']]
    .sort_values('Year')
)
```

SQL Query

```
SELECT Year, Candidate, %
FROM elections
WHERE Result = 'win'
ORDER BY Year;
```